JAVA DSA
SOLVED ALGORITHMS

VOLUME 1 — ARRAYS, STRINGS & SEARCHING PATTERNS

Concepts • Rules • Algorithm selection • Complete Java code • Dry runs • Complexity • Interview problems

This is a teaching volume, not a topic checklist.
Every included problem has a solution path and compilable Java method. The complete DSA library will be split into volumes so implementations and explanations are not sacrificed to fit one file.

Java-focused • Interview preparation • November 2026

How to use every solved problem

- Read the statement and constraints. Do not look at the code first.
- Write the brute-force idea in one or two sentences.
- Identify the repeated work or structure that can be optimized.
- State the invariant (what stays true during the loop).
- Trace the sample manually, then compare with the code.
- Re-code the method from memory and test edge cases.

Java convention: methods below are designed to be pasted into a class. Add `import java.util.*;` at the top. Unless stated otherwise, inputs are valid and arrays may be empty where the method handles it.

Complexity rules

Notation	Meaning
O(1)	Constant work independent of input size.
O(log n)	Problem size shrinks by a constant factor, e.g. binary search.
O(n)	One pass over n items.
O(n log n)	Typical efficient comparison sorting.
$O(n^2)$	All pairs / nested full scans.
Auxiliary space	Extra memory used beyond the input; include recursion stack.

1. Arrays: fundamentals and implementation

What an array is

An array stores a fixed number of elements of the same type. Elements are accessed by index in O(1). Inserting or deleting in the middle usually costs O(n), because later elements must shift.

Problem 1 — Reverse an array in place

Idea: keep two pointers at the ends. Swap them, then move inward. Stop when they meet. This changes the original array and uses O(1) extra space.

Invariant: everything outside [left,right] is already in its final reversed position.

```
static void reverse(int[] a) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;
        left++;
        right--;
    }
}
```

Dry run: [2, 7, 4, 9] → swap 2/9 → [9,7,4,2] → swap 7/4 → [9,4,7,2].

Complexity: O(n) time, O(1) extra space.

Problem 2 — Find minimum and maximum

Idea: initialize both from the first element and update them during one scan. Handle an empty array explicitly.

```
static int[] minMax(int[] a) {
    if (a.length == 0) {
        throw new IllegalArgumentException("Array is empty");
    }
    int min = a[0], max = a[0];
    for (int i = 1; i < a.length; i++) {
        if (a[i] < min) min = a[i];
        if (a[i] > max) max = a[i];
    }
    return new int[]{min, max};
}
```

Complexity: O(n) time, O(1) extra space (excluding the two-value result).

Problem 3 — Move all zeroes to the end

Idea: write every non-zero value from left to right, preserving order; then fill the remaining positions with zero. This is stable and in-place.

```
static void moveZeroes(int[] a) {
    int write = 0;
    for (int x : a) {
        if (x != 0) a[write++] = x;
    }
    while (write < a.length) a[write++] = 0;
}
```

Complexity: O(n) time, O(1) extra space.

Problem 4 — Remove duplicates from sorted array

Rule: sorted input means duplicates are adjacent. Keep a write pointer for the next unique position. Return the new logical length.

```
static int removeDuplicatesSorted(int[] a) {
    if (a.length == 0) return 0;
    int write = 1;
    for (int read = 1; read < a.length; read++) {
        if (a[read] != a[write - 1]) {
            a[write++] = a[read];
        }
    }
    return write;
}
```

Important: only the prefix [0, returnedLength) is meaningful after the method.

Complexity: O(n) time, O(1) extra space.

2. Prefix sums and difference arrays

Rule

Define prefix[0]=0 and prefix[i+1]=prefix[i]+a[i]. Then the sum of inclusive range [L,R] is prefix[R+1]-prefix[L]. Use long when sums may exceed int.

Problem 5 — Range sum queries

```
static long[] buildPrefix(int[] a) {
    long[] prefix = new long[a.length + 1];
    for (int i = 0; i < a.length; i++) {
        prefix[i + 1] = prefix[i] + a[i];
    }
    return prefix;
}

static long queryRange(long[] prefix, int left, int right) {
    return prefix[right + 1] - prefix[left];
}
```

Example: a=[3, -1, 5, 2], prefix=[0,3,2,7,9]. Sum [1,3] = 9-3 = 6.

Complexity: build O(n); each query O(1); storage O(n).

Problem 6 — Subarray sum equals K (negative numbers allowed)

Why not sliding window? Negative values mean expanding a window may decrease its sum, so the usual positive-number window rule is not reliable. Use prefix sums + frequency map.

Logic: currentPrefix - oldPrefix = k, so oldPrefix = currentPrefix-k. Count how often that earlier prefix occurred.

```
static int subarraySumEqualsK(int[] a, int k) {
    Map<Long, Integer> freq = new HashMap<>();
    freq.put(0L, 1);
    long prefix = 0;
    int answer = 0;

    for (int x : a) {
        prefix += x;
        answer += freq.getOrDefault(prefix - k, 0);
        freq.put(prefix, freq.getOrDefault(prefix, 0) + 1);
    }
    return answer;
}
```

Complexity: expected O(n) time, O(n) space. For extremely large answer counts, return long instead of int.

Problem 7 — Range increment with a difference array

Given n initially-zero elements and inclusive updates [L,R] += value, record only boundary changes. A final prefix scan materializes the result.

```
static long[] applyRangeUpdates(int n, int[][] updates) {
    long[] diff = new long[n + 1];
    for (int[] u : updates) {
        int left = u[0], right = u[1], value = u[2];
        diff[left] += value;
        if (right + 1 < n) diff[right + 1] -= value;
    }
    long[] result = new long[n];
    long running = 0;
    for (int i = 0; i < n; i++) {
        running += diff[i];
        result[i] = running;
    }
    return result;
}
```

Complexity: O(n+q), where q is the number of updates.

3. Hashing and frequency counting

When to recognize it

Use a hash map/set when you need fast expected membership checks, counts, grouping, or complement lookup. Hash operations are expected O(1), not an unconditional worst-case guarantee.

Problem 8 — Two Sum

Brute force: check all pairs, $O(n^2)$. Optimized: for each x, look for target-x among previously seen values, then store x.

```
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        int need = target - a[i];
        if (seen.containsKey(need)) {
            return new int[]{seen.get(need), i};
        }
        seen.put(a[i], i);
    }
    return new int[]{-1, -1};
}
```

Complexity: expected O(n) time and O(n) space. If integer subtraction can overflow under the constraints, calculate the complement as long.

Problem 9 — Contains duplicate

```
static boolean containsDuplicate(int[] a) {
    Set<Integer> seen = new HashSet<>();
    for (int x : a) {
        if (!seen.add(x)) return true;
    }
    return false;
}
```

Complexity: expected O(n) time, O(n) space.

Problem 10 — Group anagrams

Anagrams share the same character-frequency signature. This implementation assumes lowercase English letters.

```
static List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String word : words) {
        int[] count = new int[26];
        for (int i = 0; i < word.length(); i++) {
            count[word.charAt(i) - 'a']++;
        }
        StringBuilder key = new StringBuilder();
        for (int x : count) key.append('#').append(x);
        groups.computeIfAbsent(key.toString(), z -> new ArrayList<>())
              .add(word);
    }
    return new ArrayList<>(groups.values());
}
```

Complexity: O(total characters + 26×number of words), effectively O(total characters) for a fixed alphabet; output storage is O(total characters).

4. Two pointers

Recognition rule

Two pointers are useful when sorted order, opposite boundaries, or a fast/slow relationship lets you discard part of the search space safely.

Problem 11 — Two Sum in a sorted array

```
static int[] twoSumSorted(int[] a, int target) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        long sum = (long) a[left] + a[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

Why pointer movement is safe: if the smallest remaining value plus the largest is too small, pairing the smallest with any other remaining value is also too small; discard it. Symmetric reasoning applies when the sum is too large.

Complexity: O(n) time, O(1) space.

Problem 12 — Container With Most Water

Area = min(height[left],height[right]) × (right-left). Move the pointer at the shorter wall: moving the taller one cannot increase the limiting height and only reduces width.

```
static int maxArea(int[] h) {
    int left = 0, right = h.length - 1;
    int best = 0;
    while (left < right) {
        int width = right - left;
        int area = Math.min(h[left], h[right]) * width;
        best = Math.max(best, area);
        if (h[left] < h[right]) left++;
        else right--;
    }
    return best;
}
```

Complexity: O(n) time, O(1) space. Use long for area if constraints can exceed int.

5. Sliding window

Core rule

A sliding window represents a contiguous segment. Expand right to include new data; shrink left to restore the condition. This technique is especially straightforward when the validity condition changes monotonically as the window expands/shrinks.

Problem 13 — Longest substring without repeating characters

```
static int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char ch = s.charAt(right);
        if (last.containsKey(ch)) {
            left = Math.max(left, last.get(ch) + 1);
        }
        last.put(ch, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Invariant: the current window contains no repeated character. max() prevents left from moving backward.

Complexity: O(n) expected time, O(min(n, alphabet size)) space.

Problem 14 — Maximum sum of a fixed-size window

```
static long maxWindowSum(int[] a, int k) {
    if (k <= 0 || k > a.length) {
        throw new IllegalArgumentException("Invalid window size");
    }
    long sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    long best = sum;
    for (int right = k; right < a.length; right++) {
        sum += a[right] - a[right - k];
        best = Math.max(best, sum);
    }
    return best;
}
```

Complexity: O(n) time, O(1) extra space.

Problem 15 — Minimum size subarray sum (positive numbers)

This standard variable-window solution assumes all array elements are positive. With negative values, use a different technique such as prefix sums plus a monotonic deque for the shortest subarray with sum at least K.

```
static int minSubArrayLen(int target, int[] a) {
    int left = 0, best = Integer.MAX_VALUE;
    long sum = 0;
    for (int right = 0; right < a.length; right++) {
        sum += a[right];
        while (sum >= target) {
            best = Math.min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

Complexity: O(n) time because each pointer moves at most n times; O(1) space.

6. Kadane and subarray optimization

Problem 16 — Maximum subarray sum

At each index, either start a new subarray here or extend the best subarray ending at the previous index. This is valid even when every number is negative.

```
static int maxSubArray(int[] a) {
    if (a.length == 0) throw new IllegalArgumentException("Empty array");
    int endingHere = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        endingHere = Math.max(a[i], endingHere + a[i]);
        best = Math.max(best, endingHere);
    }
    return best;
}
```

Complexity: O(n) time, O(1) space. Use long if the sum may exceed int.

Problem 17 — Best time to buy and sell stock (one transaction)

Track the minimum price seen so far. Selling today gives profit todayPrice-minPrice.

```
static int maxProfitOneTransaction(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int best = 0;
    for (int price : prices) {
        minPrice = Math.min(minPrice, price);
        best = Math.max(best, price - minPrice);
    }
    return best;
}
```

Complexity: O(n) time, O(1) space.

7. Binary search

Template and invariant

Use binary search on sorted data or a monotonic yes/no condition. This guide uses a half-open interval [lo,hi): lo is included and hi is excluded. The answer boundary remains inside the search interval.

Problem 18 — Lower bound

Return the first index i for which a[i] >= target; if none exists, return a.length.

```
static int lowerBound(int[] a, int target) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Complexity: O(log n) time, O(1) space.

Problem 19 — Search in rotated sorted array (distinct values)

```
static int searchRotated(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;

        if (a[lo] <= a[mid]) { // left half is sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // right half is sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Complexity: O(log n) time with distinct values; duplicates can degrade worst case to O(n).

Problem 20 — Search on answer: Koko Eating Bananas

Binary search the minimum eating speed k. Feasibility is monotonic: if speed k can finish within h hours, every larger speed can too. Hours for pile p = ceil(p/k) = (p+k-1)/k, computed safely with long.

```
static int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        long hours = 0;
        for (int p : piles) {
            hours += (p + (long) mid - 1) / mid;
        }
        if (hours <= h) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Complexity: O(n log M), where M is the largest pile; O(1) extra space.

8. Sorting algorithms you must implement

Algorithm	Time	Extra space	Stable?
Insertion sort	Best O(n), average/worst O(n²)	O(1)	Yes
Selection sort	O(n²)	O(1)	Usually no
Merge sort	O(n log n)	O(n)	Yes (standard merge)
Quick sort	Average O(n log n), worst O(n²)	Expected O(log n) stack	Usually no
Heap sort	O(n log n)	O(1) iterative	No
Counting sort	O(n+k)	O(n+k)	Can be
Radix sort	O(d(n+k))	O(n+k)	Yes if stable sub-sort

Insertion sort — full implementation

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Invariant: before each i, the prefix [0,i) is sorted. Shift larger elements right and insert key into the gap. Best O(n), worst O(n²), O(1) space; stable.

Merge sort — full implementation

```
static void mergeSort(int[] a) {
    int[] temp = new int[a.length];
    mergeSort(a, temp, 0, a.length - 1);
}

static void mergeSort(int[] a, int[] temp, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(a, temp, left, mid);
    mergeSort(a, temp, mid + 1, right);
    merge(a, temp, left, mid, right);
}

static void merge(int[] a, int[] temp, int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;
    while (i <= mid && j <= right) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= right) temp[k++] = a[j++];
    for (int x = left; x <= right; x++) a[x] = temp[x];
}
```

Why: recursively sort two halves, then merge two sorted ranges in linear time. Recurrence T(n)=2T(n/2)+O(n)=O(n log n). Extra space O(n). Choosing left on equality makes the merge stable.

Quick sort — randomized pivot, Lomuto partition

```
static void quickSort(int[] a) {
    quickSort(a, 0, a.length - 1, new Random());
}

static void quickSort(int[] a, int lo, int hi, Random rnd) {
    if (lo >= hi) return;
    int pivotIndex = lo + rnd.nextInt(hi - lo + 1);
    swap(a, pivotIndex, hi);
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1, rnd);
    quickSort(a, p + 1, hi, rnd);
}

static int partition(int[] a, int lo, int hi) {
    int pivot = a[hi], store = lo;
    for (int i = lo; i < hi; i++) {
        if (a[i] <= pivot) {
            swap(a, i, store++);
        }
    }
    swap(a, store, hi);
    return store;
}

static void swap(int[] a, int i, int j) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}
```

Average expected O(n log n), worst O(n²); recursion stack expected O(log n), worst O(n). Random pivot reduces the chance of consistently bad splits but does not remove the theoretical worst case.

Heap sort — in-place max heap

```
static void heapSort(int[] a) {
    int n = a.length;
    for (int i = n / 2 - 1; i >= 0; i--) siftDown(a, n, i);
    for (int end = n - 1; end > 0; end--) {
        swap(a, 0, end);
        siftDown(a, end, 0);
    }
}

static void siftDown(int[] a, int size, int root) {
    while (true) {
        int largest = root;
        int left = 2 * root + 1, right = left + 1;
        if (left < size && a[left] > a[largest]) largest = left;
        if (right < size && a[right] > a[largest]) largest = right;
        if (largest == root) return;
        swap(a, root, largest);
        root = largest;
    }
}
```

Build heap O(n); repeated extraction O(n log n); total O(n log n), O(1) auxiliary space, not stable.

9. More interview problems to solve after these 20

Pattern	Must-practice problems
Arrays	Product Except Self; 3Sum; 4Sum; Trapping Rain Water; First Missing Positive; Majority Element; Rotate Array
Prefix/hash	Continuous Subarray Sum; Contiguous Array; Longest Consecutive Sequence; Subarray Sums Divisible by K
Two pointers	3Sum; 4Sum; Remove Duplicates II; Trapping Rain Water; Sort Colors
Sliding window	Minimum Window Substring; Permutation in String; Find All Anagrams; Longest Repeating Character Replacement; Sliding Wind
Binary search	First/Last Position; Find Peak; Kth in Sorted Matrix; Median of Two Sorted Arrays; Split Array Largest Sum
Sorting	Count Inversions; Kth Largest; Sort Colors; Merge Intervals; Meeting Rooms

10. Common mistakes checklist

- Using int when sum/product may overflow; use long after checking constraints.
- Using sliding window for arbitrary negative numbers without proving monotonicity.
- Binary search with unclear inclusive/exclusive boundaries; write the invariant first.
- Moving a pointer without proving that discarded candidates cannot be optimal.
- Updating a hash map before checking the complement in Two Sum (can reuse same index).
- Returning the whole array length when only a prefix is valid after in-place compaction.
- Not handling empty arrays, one element, duplicates, or all-negative values.
- Comparator subtraction overflow; use Integer.compare / Long.compare.
- Assuming HashMap iteration order or worst-case O(1).
- Copying code without understanding the invariant and constraints.

11. Volume roadmap

Volume	Coverage
1 (this PDF)	Arrays, strings, hashing, prefix sums, two pointers, windows, binary search, sorting
2	Linked lists, stacks, queues, deque, monotonic stack/queue, heaps
3	Recursion, backtracking, divide & conquer, greedy
4	Trees, BST, balanced trees, tries, range structures
5	Graphs: traversal, topo sort, shortest paths, MST, DSU, SCC, flow
6	Dynamic programming: core patterns through advanced optimization
7	Bit manipulation, number theory, math, geometry, randomized/string advanced
8	Core CS, Java/JVM, SQL/DBMS, OS, networks, LLD/HLD

Practice standard: for each problem, be able to state the brute-force approach, optimized pattern, invariant, Java implementation, dry run, time/space complexity, and at least three edge cases.